Solution: Matchsticks to Square

Problem Statement

You are given an integer array `matchsticks`, where `matchsticks[i]` represents the length of the i^{th} matchstick. You need to make `one square` by using these all matchsticks. You are `not allowed` to `break` any matchstick, but you can link them together, and you can use each matchstick only once.

Return `true`, if you make one square using all given matchsticks. Otherwise, return `false`.

Examples

- **Example 1**:
  - **Input**: `[5,5,5,5]`
  - **Expected Output**: `true`
  - **Justification**: The matchsticks can be arranged to form a square with each side being 5 units long, using all the matchsticks exactly once.
- **Example 2**:
  - **Input**: `[3,3,3,3,4]`
  - **Expected Output**: `false`
  - **Justification**: Despite having four matchsticks of the same length, the additional matchstick of length 4 prevents forming a perfect square.
- **Example 3**:
  - **Input**: `[1,1,2,1,2,1]`
  - **Expected Output**: `true`
  - **Justification**: These matchsticks can be arranged to form a square with each side being 2 units long (`1+1` for two sides and `2` for the other two sides).

Solution

To solve this problem, we adopt a backtracking approach which systematically explores all combinations of matchstick assignments to the sides of the square. We believe this approach is effective because it allows us to explore all potential arrangements while enabling us to backtrack if a certain arrangement does not lead to a solution. This flexibility is crucial given the nature of the problem, where the solution requires a specific distribution of matchsticks that might not be immediately apparent.

The key to this approach is to avoid redundancy and unnecessary computations by checking the feasibility of a partial solution before proceeding further. This method ensures that we only pursue promising leads, significantly reducing the search space and making the problem manageable within reasonable computational limits.

Step-by-step Algorithm

1. **Calculate the Total Length**:
   - Sum up the lengths of all the matchsticks. This total length must be divisible by 4 for a square to be possible, as all four sides of the square must have equal length.
2. **Check for Basic Validity**:
   - If the total length of matchsticks is not divisible by 4, return **false** immediately, as it's not possible to form a square.
3. **Determine Side Length**:
   - Divide the total length by 4 to find the target length for each side of the square.
4. **Sort Matchsticks**:
   - Sort the matchsticks in descending order. Starting with the longest matchsticks helps in reaching a solution faster or failing early if a solution isn't possible.
5. **Initialize Sides Array**:
   - Create an array of 4 integers, each initialized to 0, to represent the current length of each side of the square as the algorithm progresses.
6. **Backtrack to Build the Square**:
   - Starting with the first matchstick, attempt to place each matchstick on each side of the square one by one.
   - For each matchstick:
     - Try adding it to each side (one side at a time).
     - If adding the matchstick to a side does not exceed the target side length, recursively attempt to place the next matchstick.
     - If a matchstick cannot be placed on any side without exceeding the target length, backtrack: remove the last matchstick placed and try the next option.
   - If all matchsticks are successfully placed such that all four sides are of equal length, return **true**.
   - If some matchsticks cannot be placed successfully after trying all options, return **false**.
7. **Success or Failure**:
   - The algorithm returns **true** if a combination that forms a square is found, otherwise **false**.

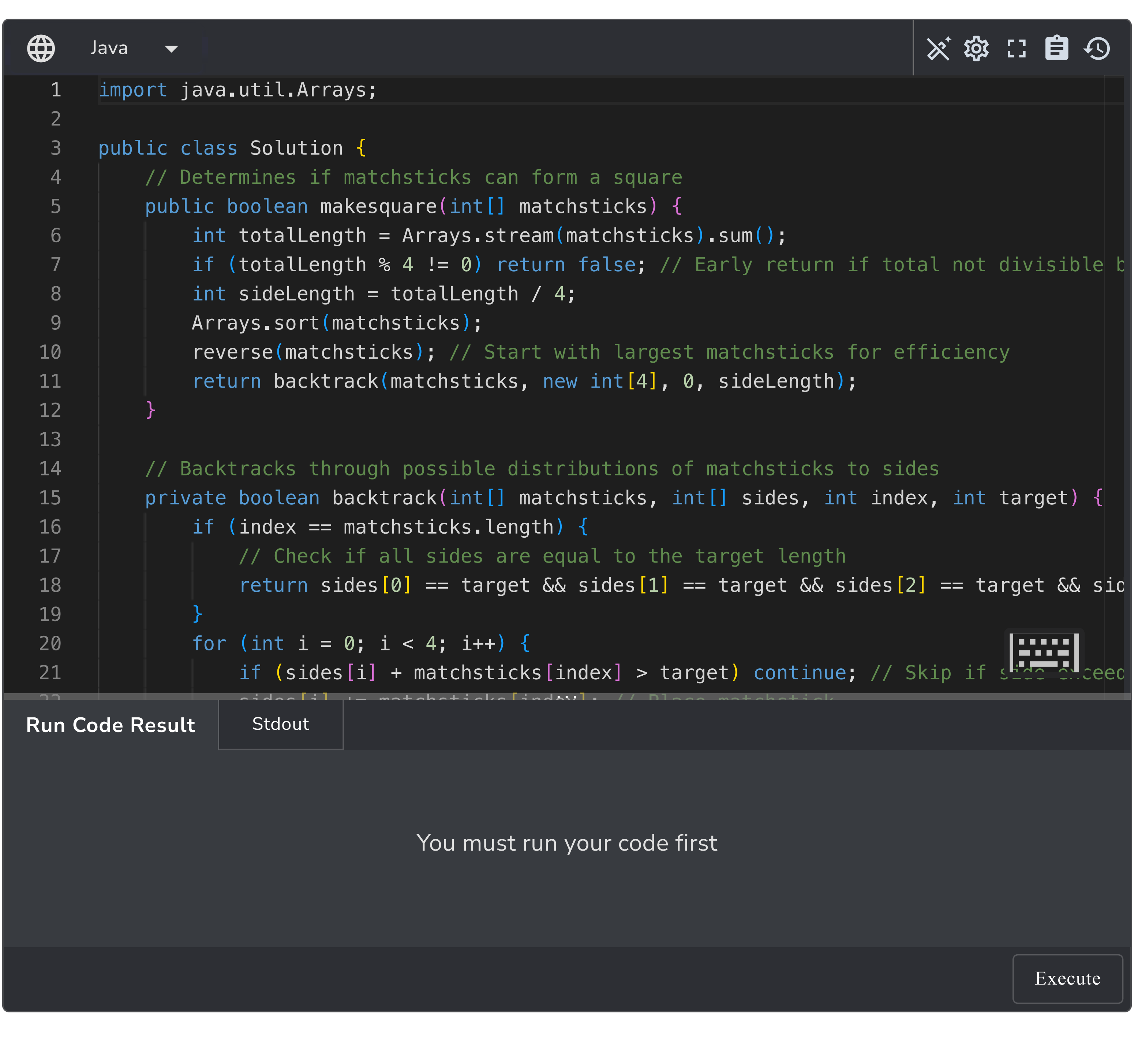
Algorithm Walkthrough

Let's consider the input `[1,1,2,1,2,1]`.

1. **Total Length Calculation**:
   - First, calculate the total length of all matchsticks: `1 + 1 + 2 + 1 + 2 + 1 = 8`.
2. **Check for Basic Validity**:
   - Verify if the total length is divisible by 4: `8 % 4 == 0`, which is true. Therefore, a square might be possible.
3. **Determine Side Length**:
   - Calculate the target length for each side of the square: `8 / 4 = 2`.
4. **Sort and Reverse Matchsticks**:
   - Sort the matchsticks in ascending order: `[1, 1, 1, 2, 2, 1]`, then reverse the array to start with the largest matchsticks for efficiency: `[2, 2, 1, 1, 1, 1]`.
5. **Backtracking to Place Matchsticks**:

- **First Matchstick (2)**:
  - Place the first matchstick (2) on the first side, making the sides array `[2, 0, 0, 0]`.
  - Call the backtrack function recursively to attempt placing the next matchstick.
- **Second Matchstick (2)** in the Backtrack Function:
  - The first iteration of the loop skips the first side because `2 + 2 > 2` (exceeds target).
  - In the second iteration of the loop, place the second matchstick (2) on the second side, updating the sides array to `[2, 2, 0, 0]`.
  - Recursively call the backtrack function to attempt placing the next matchstick.
- **Third Matchstick (1)**:
  - For the third matchstick, the first side is full, and the second side is also full. The third iteration places the matchstick on the third side, `[2, 2, 1, 0]`.
  - Recursively call the backtrack function.
- **Fourth Matchstick (1)**:
  - For the fourth matchstick, the first two sides are full, and the third side already has a matchstick but the sum of sides[i]+ matchsticks[i] is less than or equal to `target(2)`. The fourth matchstick is placed on the third side, updating the sides array to `[2, 2, 2, 0]`.
  - Recursively call the backtrack function.
- **Fifth Matchstick (1)**:
  - For the fifth matchstick, the first side, second side, and third is full. The fourth iteration places the matchstick on the fourth side, `[2, 2, 2, 1]`.
  - Recursively call the backtrack function.
- **Sixth Matchstick (1)**:
  - Finally, the sixth matchstick is placed on the fourth side, completing all sides to the target length. The sides array is updated to `[2, 2, 2, 2]`.
  - With all sides equal to the target length, the condition for forming a square is met.
- **Return True**:
  - Since the algorithm successfully placed all matchsticks without any side exceeding the target length and all sides equal to the target length, it returns `true`.

Code



```java
import java.util.Arrays;

public class Solution {
    // Determines if matchsticks can form a square
    public boolean makesquare(int[] matchsticks) {
        int totalLength = Arrays.stream(matchsticks).sum();
        if (totalLength % 4 != 0) return false; // Early return if total not divisible b
        int sideLength = totalLength / 4;
        Arrays.sort(matchsticks);
        reverse(matchsticks); // Start with largest matchsticks for efficiency
        return backtrack(matchsticks, new int[4], 0, sideLength);
    }

    // Backtracks through possible distributions of matchsticks to sides
    private boolean backtrack(int[] matchsticks, int[] sides, int index, int target) {
        if (index == matchsticks.length) {
            // Check if all sides are equal to the target length
            return sides[0] == target && sides[1] == target && sides[2] == target && sic
        }
        for (int i = 0; i < 4; i++) {
            if (sides[i] + matchsticks[index] > target) continue; // Skip if side exceed
```

Complexity Analysis

Time Complexity

- **Worst-case scenario**: $O(4^N)$, where N is the number of matchsticks. In the worst case, the algorithm tries to place each matchstick in each of the four possible sides, leading to a recursive call tree of depth N with a branching factor of 4.
- **Sorting**: $O(N \log N)$ for sorting the matchsticks in descending order, which is a preprocessing step.

The overall time complexity is dominated by the backtracking process, making it $O(4^N)$ in the worst case.

Space Complexity

- **Recursive Call Stack**: $O(N)$, where N is the number of matchsticks. This is the maximum depth of the recursive call stack.
- **Additional Space**: $O(N)$ for storing the matchsticks and the sides array.

The overall space complexity is $O(N)$, considering both the space for the input and the recursive call stack.

← Previous
Matchsticks To Square (Medium)
Next →
Score Of Parentheses (Medium)
☐ Mark as Completed